

FASES DE DESENVOLVIMENTO (GUIA)

Fase 1: Alinhamento e Configuração Inicial (Dia 1)

Antes de escrever qualquer linha de código, os programadores e você precisam alinhar a estrutura básica para que ninguém trabalhe no escuro.

1. **Definição do Banco de Dados:** O Iarley/Francisco e o Matheus precisam criar o arquivo SQLite (.db). Definam uma tabela simples chamada produtos com as colunas: id (PK), nome (TEXT), quantidade (INTEGER) e preco (REAL).
2. **Setup do Repositório/Ambiente:** Decidam se vão usar um repositório Git ou apenas compartilhar o arquivo .py. Como o projeto exige **Tela Única (Single Window)** com tkinter, o código pode ficar centralizado em um único arquivo principal para facilitar a agilidade.

Fase 2: O Plano de Desenvolvimento (Para os Programadores)

O Iarley/Francisco e o Matheus devem dividir as tarefas para não baterem cabeça. Uma sugestão eficiente de divisão para eles é:

- **Programador 1 (Interface e Eventos):** Foca em construir a janela do tkinter, posicionar os campos de texto (Entry) para Nome, Quantidade e Preço, os botões (Button) e a tabela de exibição (Treeview). Também implementa a lógica do **Update** (clicar na linha da Treeview e jogar os dados de volta para os campos de texto).
- **Programador 2 (Banco de Dados e Log):** Foca em criar as funções de backend: conectar ao banco, as queries SQL do CRUD (Insert, Select, Update, Delete), e a função de persistência que escreve no log_estoque.txt usando o modo 'a'.

Fase 3: O Seu Plano de Ação (Saymon - QA)

Como QA, seu trabalho começa *antes* de o programa ficar pronto. Enquanto eles programam, você vai preparar o terreno.

1. Criar a Massa de Dados com Faker

O relatório de vocês precisa comprovar o uso de dados fictícios para testes. Você mesmo pode criar um script rápido em Python usando a biblioteca Faker para gerar, por exemplo, 50 produtos aleatórios e inseri-los no banco de dados SQLite de teste.

Isso vai ajudar a ver se a Treeview (a grade) aguenta paginação/scroll e se a exibição está correta.

2. Preparar a Planilha de Casos de Teste

Você deve testar exaustivamente os requisitos do professor. Foque em encontrar falhas nestes pontos críticos:

- **Validação de Tipos (Try/Except):** O que acontece se você digitar "dez" no campo de quantidade e clicar em salvar? O sistema deve exibir um alerta (messagebox) e **não pode travar (crash)**.
- **Limpeza de Strings (.strip()):** Insira um produto com espaços no início e no fim (ex: " Mouse sem Fio "). Verifique no banco se o sistema limpou esses espaços antes de salvar.
- **Confirmação de Exclusão:** Ao clicar em deletar, o messagebox realmente pergunta "Tem certeza?" antes de apagar?
- **Verificação do Log:** A cada clique de salvar, editar ou deletar, abra o arquivo log_estoque.txt e veja se a linha foi adicionada exatamente no formato que o professor exigiu (com data, hora e a descrição da ação).

Fase 4: O Relatório RAD (Guilherme Gomes)

O Guilherme precisa acompanhar o processo desde o início para documentar a experiência. Ele deve focar em responder aos três pilares pedidos no trabalho:

1. **Decisões Rápidas:** Explicar por que escolheram o tkinter (já vem nativo no Python, acelera a GUI) e o SQLite (banco em arquivo leve, sem necessidade de configurar servidores complexos).
2. **Estratégia de Teste:** Documentar como você (Saymon) utilizou a biblioteca Faker para estressar o banco de dados inicial e como validou os logs.
3. **Visão de Futuro (Versão 2.0):** Sugerir duas funcionalidades de negócio para o próximo ciclo. *Dica: sugestões como "Gráfico de relatórios de produtos mais vendidos" ou "Sistema de autenticação/login de usuários" encaixam perfeitamente aqui.*

Checklist de Entrega (O que você, como QA, deve validar no final)

Antes de compactarem o arquivo .zip ou .rar no padrão Nome_Sobrenome_RAD.zip, certifique-se de que nada ficou para trás:

- [] O script principal Python (.py ou .ipynb) roda direto sem erros de sintaxe.
- [] O arquivo do banco de dados (.db) contém dados iniciais populados.
- [] O relatório do Guilherme está em PDF e tem entre 1 e 2 páginas.
- [] O arquivo log_estoque.txt está presente com o histórico dos testes que você realizou.
- [] A aplicação roda **estritamente em uma única tela** (sem abrir pop-ups para edição).

Funções

1. Francisco (Programador Backend & Banco de Dados)

Sua Missão: Garantir que os dados entrem, saiam e sejam modificados no banco de dados de forma segura, além de manter o histórico de acessos.

O que deve fazer (Passo a Passo):

- **Criar o Banco de Dados:** Escrever a função que cria o arquivo SQLite (.db) e a tabela de produtos caso ela não exista. As colunas obrigatórias são: id, nome, quantidade e preco.
- **Criar a Função de Log (Modo 'a'):** Desenvolver uma função chamada salvar_no_log(acao, detalhes). Ela deve abrir o arquivo log_estoque.txt obrigatoriamente usando o modo de adição open('log_estoque.txt', 'a') para que os testes anteriores não sejam apagados. Deve capturar a data e hora atual do sistema.
- **Desenvolver as Queries do CRUD:** Criar funções em Python que recebam os dados da interface e executem os comandos SQL (INSERT, UPDATE, DELETE, SELECT).

- **Tratamento de Erros:** Proteger as funções de inserção e atualização com blocos try/except. Se o usuário digitar um texto onde deveria ser um número, a função deve retornar um aviso/erro em vez de quebrar o programa.

2. Iarley (Programador Frontend & Interface Tkinter)

Sua Missão: Construir a casca visual do programa, garantindo que tudo funcione em uma única janela.

O que deve fazer (Passo a Passo):

- **Desenhar a Tela Única (Single Window):** Criar a janela principal com tkinter. É **proibido** criar janelas secundárias ou pop-ups de formulário.
- **Criar o Formulário (Entries e Labels):** Posicionar os campos de texto (Entry) para que o usuário digite o Nome, Quantidade e Preço do produto.
- **Criar os Botões de Ação:** Adicionar os botões "Cadastrar", "Atualizar" e "Excluir".
- **Configurar a Tabela (Treeview):** Criar a grade visual (ttk.Treeview) para listar os produtos com as colunas alinhadas.
- **Lógica do Clique (Update):** Configurar o evento de clique na Treeview (tree.bind("<ButtonRelease-1>", ...)). Quando o usuário clicar em uma linha da tabela, os dados desse produto devem reaparecer automaticamente nos campos de texto lá de cima para edição.
- **Mensagem de Confirmação:** No botão de excluir, configurar para abrir um messagebox.askyesno perguntando se o usuário realmente deseja deletar o item antes de chamar o backend.

3. Saymon (QA - Analista de Qualidade)

Sua Missão: Atuar como a "linha de defesa" do projeto, testando os limites do programa e garantindo que ele atenda 100% dos critérios do professor para ganhar nota máxima.

O que deve fazer (Passo a Passo):

- **Criar o Script de População (Faker):** Criar um arquivo rápido chamado popular_banco.py. Instalar a biblioteca Faker (pip install faker) e fazer um loop que insere automaticamente uns 20 produtos falsos no banco do francisco. Isso vai provar a estratégia de teste do grupo.

- **Validar a Limpeza de Dados (.strip()):** Testar cadastrar produtos cheios de espaços em branco antes e depois do nome (ex: " Notebook "). Verificar se o sistema limpa esses espaços antes de salvar.
- **Testar Quebra do Sistema (Inputs Inválidos):** Tentar colocar letras no preço ("quinhentos") ou decimais errados na quantidade. O sistema precisa exibir um alerta amigável na tela e continuar funcionando normalmente.
- **Auditar o Arquivo de Log:** Abrir o log_estoque.txt após os testes e verificar se o formato das linhas confere exatamente com o padrão exigido.
- **Reportar Falhas:** Se encontrar qualquer comportamento estranho, preencher uma listinha simples e mandar para o Matheus ou Iarley corrigirem antes da entrega.

4. Guilherme Gomes (Documentação RAD)

Sua Missão: Observar o fluxo de trabalho da equipe e redigir o relatório em PDF (de 1 a 2 páginas) comprovando a aplicação da metodologia ágil.

O que deve fazer (Passo a Passo):

- **Seção 1 - Decisões Rápidas:** Explicar por que a equipe escolheu Python, Tkinter e SQLite para acelerar o desenvolvimento do protótipo (foco no RAD). Destacar que a escolha de uma Tela Única evitou a perda de tempo gerenciando múltiplas janelas.
- **Seção 2 - Estratégia de Teste:** Descrever detalhadamente como o Saymon utilizou a biblioteca Faker para gerar dados fictícios no banco e como foram feitos os testes de estresse (erros de digitação e persistência do log).
- **Seção 3 - Visão de Futuro (Versão 2.0):** Como o RAD é focado em ciclos rápidos e melhoria contínua, o Guilherme deve sugerir duas novas funções de negócio para o próximo ciclo. *Exemplos: 1. Alerta visual de estoque baixo (cor vermelha na Treeview); 2. Sistema de autenticação/login de funcionários.*
- **Gerar o PDF:** Formatar esse texto de maneira bem profissional e exportar em formato .pdf.

Como juntar tudo:

Quando o Francisco acabar as funções do banco e o Iarley acabar o design da tela, eles devem abrir uma chamada de vídeo e criar o arquivo final main.py.

1. Eles colam as funções do Francisco no topo.
2. Colam a interface do Iarley embaixo.

3. O Iarley faz os botões chamarem as funções do Francisco (ex: `command=inserir_produto_no_banco`).
4. O arquivo final unificado é passado para o **Saymon** fazer a bateria final de testes e gerar o `log_estoque.txt` final.
5. Com tudo aprovado, vocês compactam a pasta no formato `Nome_Sobrenome_RAD.zip` contendo os 4 arquivos obrigatórios.